

CLEANROOMX

Engineering User & Validation Manual

Design • Simulation • HVAC • Networks • BIM/IFC • Verification • Evidence • Controlled Handoff

Document	Engineering User & Validation Manual
Revision	2.1
Stable release	v0.102.1
Engineering baseline	Stable release baseline; development-preview features explicitly labeled
Manual date	1 October 2026

Engineering and regulatory boundary

CleanroomX provides engineering screening, simulation, verification, and software/provenance evidence. It does not by itself establish cleanroom certification, CFD validation, commissioning/TAB acceptance, manufacturer approval, or regulatory compliance. Applicable criteria remain explicit project inputs and require qualified review.

Intended audience: cleanroom/HVAC engineers, BIM coordinators, verification engineers, reviewers, QA/validation personnel, technical leads, and controlled-document owners.

This revision reorganizes the manual around operator tasks, review gates, and industrial use rather than repository chronology. Development status is isolated in the final appendix.

Contents

- 0. Document control 6**
 - Revision history 6
 - Document approval and controlled-copy rules 7
 - Software status convention used in this manual 7
- 1. Purpose and intended use 8**
 - 1.3 Operator quick path 8
 - 1.1 What CleanroomX should be used for 8
 - 1.2 What CleanroomX must not be represented as doing by itself 9
- 2. Roles and responsibilities 10**
 - 2.1 Minimum project data pack 10
- 3. Release and qualification policy 12**
 - 3.1 Recommended production baseline 12
 - 3.2 Development preview isolation 12
 - 3.3 Change-control rule 12
 - 3.4 Units, rounding and numerical-use convention 12
- 4. Installation and readiness 14**
 - 4.1 Windows repository launch 14
 - 4.2 Standard Python installation 14
 - 4.3 Installation acceptance checklist 14
- 5. File and data governance 15**
 - 5.1 Project document 15
 - 5.2 Strict JSON rules 15
 - 5.3 Save integrity 15
 - 5.4 Legacy migration 15

- 5.5 Relative references 16
- 6. Standard operating procedure SOP-01: start a project 17**
- 7. Standard operating procedure SOP-02: 2D/3D spatial design 18**
 - 7.1 Spatial model 18
 - 7.2 Recommended sequence 18
 - 7.3 Spatial validation checks 18
 - 7.4 Undo/redo discipline 18
- 8. Standard operating procedure SOP-03: synchronize spatial and engineering models . . . 19**
 - 8.1 Synchronization rule 19
 - 8.2 Push to analysis 19
 - 8.3 Pull from analysis 19
 - 8.4 Synchronization states 19
- 9. Standard operating procedure SOP-04: validate, run and review an analysis 20**
 - 9.1 Freshness protection 20
 - 9.2 Abandon behavior 20
- 10. Room and project verification 21**
 - 10.1 Room volume 21
 - 10.2 Nominal supply ACH 21
 - 10.3 Pressure-cascade checks 21
 - 10.4 Aggregate result states 21
- 11. Particle decay and recovery screening 22**
- 12. Psychrometrics, thermal and HVAC workflows 23**
 - 12.1 Steady airflow balance 23
 - 12.2 Preliminary fan duty 23
 - 12.3 Thermal uncertainty 23

- 13. Duct and airflow network analysis 24**
 - 13.1 Engineering review points 24
 - 13.2 Numerical integrity 24
- 14. Fan/system operating-point studies 25**
 - 14.1 Required engineering checks 25
 - 14.2 Cross-study consistency 25
- 15. Uncertainty and scenario studies 26**
- 16. Engineering consistency, dossier and assurance 27**
 - 16.1 Consistency checks 27
 - 16.2 Engineering dossier 27
 - 16.3 Design assurance 27
 - 16.4 Assurance snapshots 27
- 17. ProofGraph - development preview 28**
- 18. BIM / IFC interoperability 29**
 - 18.1 BIM operating rules 29
 - 18.2 Native IFC dependency 29
- 19. Persistence, autosave, recovery and revisions 30**
 - 19.1 Atomic persistence 30
 - 19.2 Autosave and crash recovery 30
 - 19.3 Saved revisions 30
- 20. Diagnostics and health checks 31**
- 21. Reporting and evidence handoff 32**
 - 21.1 Report types 32
 - 21.2 Report review checklist 32
 - 21.3 Portable project bundle 32

22. Command-line reference 33

23. Industrial workflow examples 35

23.1 Example A - new cleanroom design package 35

23.2 Example B - design change after commissioning feedback 35

23.3 Example C - external review handoff 35

24. Troubleshooting and failure handling 37

24.1 Fail-safe rule 37

24.2 Deviation and exception control 37

25. Security and deployment 39

25.1 Security principles 39

25.2 Deployment 39

26. Verification and validation strategy for organizations 40

27. External standards and regulatory references 41

28. Controlled release checklist 42

28.1 Engineering traceability matrix 42

Baseline 42

Inputs 42

Execution 42

Engineering review 43

Evidence 43

Appendix A - recommended project folder structure 44

Appendix B - operator quick reference 45

Appendix C - glossary 46

Appendix D - controlled source map 47

CHAPTER 0

Document control

0. Document control

Field	Controlled value
Document title	CleanroomX Professional Engineering User Manual & Verification Handbook
Document ID	CX-UM-001
Revision	2.1
Issue date	1 October 2026
Document class	Controlled engineering operating reference
Approval record	Maintained by the adopting organization's document-control process
Stable software baseline	v0.102.1
Stable release validation	1008 passing tests on Python 3.11, 3.12 and 3.13 at the final deterministic-spatial-drag release gate
Project schema	cleanroomx.project, schema version 1
Base package dependencies	None required by the package metadata
Optional BIM dependency	ifcopenshell via cleanroomx[bim]
Primary desktop command	cleanroomx-gui
Primary project check	cleanroomx-project-check
Primary project execution	cleanroomx-project-run
Primary evidence handoff	cleanroomx-project-bundle
Assurance snapshot tool	cleanroomx-assurance-snapshot

Revision history

Rev.	Date	Description
2.1	2026-10-01	Added controlled-copy rules, operator decision gates, minimum project data-pack requirements, units/numerical conventions, deviation control, change-impact guidance, traceability matrix, and expanded official standards references.
2.0	2026-09-30	Rebuilt the prior solution manual into an operator-focused, industry-style controlled manual with SOPs, release-status rules, acceptance gates, evidence handling, troubleshooting, administration, and engineering boundaries.

Document approval and controlled-copy rules

This manual is an engineering operating reference, not an approval signature. Organizations adopting CleanroomX should record prepared by, technical review, QA/document-control review, approval, effective date, and superseded revision in their own authorized document-control system.

Controlled-use rules:

- verify the manual revision against the approved software baseline before use;
- do not use an uncontrolled local copy as the sole basis for a released engineering package;
- preserve superseded manuals when required for reconstruction of historical work;
- record deviations from this manual in the project's change/deviation record;
- keep project approval signatures outside CleanroomX unless an organization has separately implemented an authorized electronic-signature process.

A repository commit, SHA-256 digest, or successful CI run demonstrates software/document identity and automated test status. It does not identify or authenticate the human approver of a project.

Software status convention used in this manual

RELEASED means the behavior is part of v0.102.1 and is supported by the release evidence. DEVELOPMENT PREVIEW means the capability exists on current main after v0.102.1 but is not part of the stable release baseline. PENDING means code exists only in an open pull request and is excluded from the operational baseline.

For production or regulated work, use a pinned release/tag or an internally approved commit. Do not treat a moving development branch as a qualified baseline.

CHAPTER 1

Purpose and intended use

1. Purpose and intended use

CleanroomX provides a common engineering environment for cleanroom spatial design, airflow/HVAC calculations, pressure and fan/network studies, cleanroom verification screening, uncertainty studies, reproducible project execution, reporting, and auditable numerical provenance.

This manual is written for engineers, verification specialists, project reviewers, QA/validation personnel, BIM coordinators, technical administrators, and developers who need a repeatable operating method rather than a repository walkthrough.

The intended operating pattern is:

requirements -> controlled inputs -> model validation -> engineering run -> result review -> consistency/assurance checks -> report/evidence export -> independent approval.

1.3 Operator quick path

For day-to-day use, the minimum controlled path is:

Gate	Operator action	Release condition
G0 - Baseline	Identify CleanroomX version/commit and Python/runtime	Approved baseline is known and reproducible
G1 - Inputs	Confirm source documents, units, assumptions, criteria and mappings	Inputs are complete enough for the intended analysis
G2 - Validate	Run the workflow validation path	No unresolved validation errors
G3 - Execute	Run the analysis without changing its inputs mid-run	Run completes with an understood solver/result state
G4 - Review	Inspect normalized result, diagnostics/provenance and human-readable report	Result is fresh, plausible and traceable
G5 - Cross-check	Perform independent hand checks and applicable consistency/assurance checks	No unexplained contradictions or unresolved critical findings
G6 - Package	Generate report, bundle and/or assurance snapshot as applicable	Evidence is bound to the approved revision
G7 - Approve	Independent engineering/quality approval outside the calculation engine	Package is formally released by the responsible organization

Stop rule: if any gate is not satisfied, do not skip forward by interpreting an error, stale result, unknown state or integrity failure as acceptable.

1.1 What CleanroomX should be used for

- Preliminary and detailed engineering calculations supported by the implemented models.
- Repeatable room, HVAC, duct, branch, loop, pressure and fan studies.

- Comparison of design requirements with calculated or supplied engineering evidence.
- Controlled project files with guarded persistence, revisions, recovery, and run evidence.
- Engineering dossiers, project bundles, portable reports, diagnostics, and deterministic snapshots.
- 2D/3D spatial design coordinated with engineering room dimensions.
- Reproducible batch execution and CLI automation.

1.2 What CleanroomX must not be represented as doing by itself

- Issuing ISO cleanroom certification.
- Replacing field particle-count classification or monitoring plans.
- Replacing commissioning, testing, adjusting and balancing (TAB), validation protocols, or authority inspections.
- Replacing CFD where local airflow distribution, turbulence, contamination transport, thermal plumes, or detailed velocity fields are required.
- Replacing manufacturer fan/filter/equipment selection and approval.
- Automatically interpreting every external standard or regulatory requirement.
- Establishing digital signer identity merely because a SHA-256 content digest matches.

CHAPTER 2

Roles and responsibilities

2. Roles and responsibilities

A professional workflow should assign responsibilities explicitly.

Role	Minimum responsibility
Project engineer	Owens design inputs, units, assumptions, acceptance criteria, and engineering interpretation.
Cleanroom/HVAC engineer	Reviews room airflows, ACH, pressure cascade, HVAC loads, duct/network and fan results.
BIM/spatial coordinator	Maintains room geometry, spatial identity, IFC references and synchronization mappings.
Verification/validation reviewer	Confirms the correct model was run, evidence is fresh, criteria are appropriate, and exceptions are resolved.
QA/document control	Controls approved software baseline, manual revision, exported records and retention rules.
System administrator	Maintains Python/runtime installation, file permissions, backups, deployment and update process.
Approver/authority	Provides final project acceptance outside the software.

Segregation of duties recommendation: for high-consequence work, the person approving a final engineering package should not rely only on the same unchecked input preparation that produced it. Independent review of inputs, assumptions, units, source documents, and critical outputs is recommended.

2.1 Minimum project data pack

Before a project is treated as ready for controlled analysis, the responsible engineer should identify the source for each applicable item.

Data class	Typical controlled source	Required review
Room geometry	Approved drawings, BIM/IFC model, survey	Dimensions, elevations, room identity
Cleanliness/classification criteria	Approved project requirement and applicable standard	Edition, occupancy state, particle sizes, limits
Pressure cascade	Approved room-pressure philosophy / URS / design basis	Reference room, sign convention, target differentials
Airflow and ACH	Design air schedule, TAB basis, measured data	Supply/return/exhaust basis and units
HVAC/thermal inputs	Load schedule, psychrometric design basis	Design conditions, safety factors, units
Duct/network data	Duct schedule, fittings, branch topology	Geometry, roughness/loss coefficients, topology
Fan data	Manufacturer-approved curve or qualified test data	Speed, density basis, interpolation domain
Leakage/pressure-network data	Test data or approved engineering assumptions	Model form, coefficient, exponent/orifice properties

Data class	Typical controlled source	Required review
Uncertainty bounds	Measurement specification or approved engineering basis	Distribution/bounds and correlation assumptions
Acceptance criteria	Controlled requirement/rule pack	Applicability, revision and approval
External evidence files	Approved source documents/data exports	Revision, file identity and path stability

Missing data must be treated as a documented limitation, not silently replaced by a default assumption unless that default is explicitly authorized and recorded.

CHAPTER 3

Release and qualification policy

3. Release and qualification policy

3.1 Recommended production baseline

Use CleanroomX v0.102.1 for a controlled stable baseline unless your organization has separately qualified a later commit.

The v0.102.1 release evidence records 1008 passing tests on each supported Python version (3.11, 3.12, 3.13) at the final deterministic spatial-drag release gate, together with launcher, wheel-install, GUI smoke, spatial regression, autosave race, and solver/provenance compatibility checks.

3.2 Development preview isolation

Current main contains additional unreleased engineering/evidence work beyond v0.102.1. Examples documented on main include ProofGraph foundations, pressure-design evidence integrations, additional evidence adapters, room pressure/leakage network work, and design-foundation/assurance enhancements.

These features may be technically valuable, but they must be labeled DEVELOPMENT PREVIEW until incorporated into a released or internally qualified baseline.

3.3 Change-control rule

Before a controlled project begins, record:

- CleanroomX version or exact commit SHA;
- Python version;
- operating system/runtime context;
- project-file revision/digest if used in your quality system;
- rule-pack/evidence revisions where applicable;
- external source documents and manufacturer data revisions.

Do not silently upgrade the software in the middle of an approved verification package. If the software baseline changes, rerun the affected analyses and document the change.

3.4 Units, rounding and numerical-use convention

CleanroomX workflows are built around explicit engineering units defined by their input contracts. The operator is responsible for confirming that each source value has been converted to the unit expected by the selected workflow.

For controlled calculations:

- store and calculate with the canonical numerical state provided by the workflow;
- treat rounded values in tables, reports and plots as presentation unless the workflow explicitly defines them as an input;

- do not copy a rounded displayed result back into a downstream calculation when a canonical machine-readable result is available;
- record any manual unit conversion used to create a controlled input;
- verify sign conventions for pressure, flow direction and reference nodes before interpreting results;
- use an independent reasonableness calculation for critical outputs.

CleanroomX development explicitly protects several composed workflows from feeding presentation rounding back into engineering calculations. That implementation protection does not remove the operator's responsibility to check source units and physical meaning.

CHAPTER 4

Installation and readiness

4. Installation and readiness

4.1 Windows repository launch

From PowerShell:

```
cd C:\CleanroomX
.\start-cleanroomx.ps1 --check
.\start-cleanroomx.ps1
```

Command Prompt / Explorer-compatible launcher:

```
start-cleanroomx.cmd
```

4.2 Standard Python installation

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# Linux/macOS
# source .venv/bin/activate
```

```
python -m pip install -e .
cleanroomx-gui --check
cleanroomx-gui --demo
```

For development and repository validation:

```
python -m pip install -e .[dev]
python -m pytest -q
```

For optional native IFC support:

```
python -m pip install -e .[bim]
```

4.3 Installation acceptance checklist

Do not begin controlled project work until all applicable items pass:

- [] Correct Python version is active.
- [] `cleanroomx-gui --check` returns readiness successfully.
- [] The demo project opens.
- [] 2D and 3D views render without application errors.
- [] A sample analysis validates and runs.
- [] The intended project folder is writable.
- [] Backup/retention location is available.
- [] Optional IFC functionality is tested if required.
- [] The software version/commit is recorded in the project record.

CHAPTER 5

File and data governance

5. File and data governance

5.1 Project document

The desktop project format is strict JSON using:

- schema: cleanroomx.project
- schema version: 1

A typical structure is:

```
{
  "schema": "cleanroomx.project",
  "schema_version": 1,
  "application_version": "0.102.1",
  "project": {
    "name": "Facility A - Cleanroom Upgrade",
    "metadata": {}
  },
  "analyses": [
    {
      "id": "facility-verification",
      "name": "Facility verification",
      "kind": "project_verification",
      "input": {}
    }
  ],
  "active_analysis_id": "facility-verification"
}
```

5.2 Strict JSON rules

Engineering ingestion is intentionally fail-closed. Malformed JSON, unsupported future schema versions, duplicate analysis identifiers, invalid active-analysis references, non-finite values such as NaN or Infinity, and other invalid contract states are rejected rather than silently normalized into an uncertain result.

5.3 Save integrity

Project persistence uses validated, atomic save behavior. The application also includes external-write protection so a file changed by another process or another CleanroomX instance is not silently overwritten by a stale in-memory copy.

5.4 Legacy migration

Supported legacy single-analysis formats can be migrated into schema version 1. A migrated file is treated as a converted unsaved copy. The first save must use a new destination rather than destructively replacing the legacy source.

5.5 Relative references

File-backed workflows such as dossiers and consistency checks may reference external engineering files. Relative paths are interpreted in the project context. When a project is moved or saved to a new location, confirm every relative dependency still resolves to the intended source revision.

CHAPTER 6**Standard operating procedure
SOP-01: start a project****6. Standard operating procedure SOP-01: start a project**

Purpose: create a traceable CleanroomX project before calculations begin.

Procedure

1. Confirm the approved software baseline and run `cleanroomx-gui --check`.
2. Create a new project or copy an approved project template.
3. Enter a unique project name and controlled metadata required by your organization.
4. Save the project to the controlled working directory before adding file-backed analyses.
5. Create the required analysis records with clear names and stable IDs.
6. Enter engineering inputs in SI units expected by the selected workflow.
7. Run Validate before Run.
8. Resolve validation errors before proceeding.
9. Save the project after the first successful validated setup.
10. Record the initial project revision in the project log or document-management system.

Acceptance gate

A project is ready for engineering execution only when the file is saved, inputs validate, required source references resolve, and the software baseline is recorded.

CHAPTER 7

Standard operating procedure SOP-02: 2D/3D spatial design

7. Standard operating procedure SOP-02: 2D/3D spatial design

The 2D editor and 3D viewer share one canonical spatial model. The 3D view is not a separate copy of the design.

7.1 Spatial model

A project may contain floors, rooms, dimensions, elevations, classifications, openings, devices/equipment, pressure display values, view state, and engineering-room mappings.

7.2 Recommended sequence

1. Define floor(s), reference elevation and ceiling height.
2. Create rooms with unique, stable names and realistic dimensions.
3. Place rooms using the 2D workspace and confirm no unintended overlap.
4. Add doors, windows, transfer openings, supply/return/exhaust elements, FFUs, equipment and sensors as needed.
5. Use grid snapping only when its spacing matches the project drafting convention.
6. Run spatial validation.
7. Review the same model in 3D to catch elevation, height and placement errors.
8. Establish engineering mappings only after room identity is stable.
9. Save a controlled revision before major geometry restructuring.

7.3 Spatial validation checks

CleanroomX checks for conditions including invalid or non-positive dimensions, non-finite coordinates, duplicate stable IDs, duplicate room names, room overlap, dangling room references, orphan/unassigned devices, devices outside assigned rooms, elevation violations, unsupported device types, and malformed view state.

A spatial validation pass means the software model is internally consistent. It does not prove process flow, GMP zoning, egress, fire protection, architectural code compliance, CFD performance, or cleanroom classification.

7.4 Undo/redo discipline

Project-wide transactional Undo/Redo is designed for operator edits. Use it to correct inadvertent editing, but do not treat Undo/Redo as a substitute for controlled saved revisions or an external document-management system.

CHAPTER 8**Standard operating procedure
SOP-03: synchronize spatial and****8. Standard operating procedure SOP-03:
synchronize spatial and engineering models**

CleanroomX deliberately separates spatial geometry from engineering input/evidence.

8.1 Synchronization rule

For supported room-verification/project-verification mappings, synchronization is dimension-only: length, width and height. Spatial X/Y placement is not rewritten by pulling engineering dimensions, and verification pressure is not written back as design geometry.

8.2 Push to analysis

Use Push to analysis when the spatial model is the approved geometry source.

Before pushing:

- confirm room mappings;
- confirm no unresolved overlap/identity errors;
- confirm dimension units;
- save the project.

After pushing, revalidate and rerun the affected engineering analysis.

8.3 Pull from analysis

Use Pull from analysis when the engineering input dimensions are the approved source.

Pulling dimensions preserves room plan location. Review 2D/3D after the pull to confirm the resulting geometry is acceptable.

8.4 Synchronization states

State	Engineering meaning
synchronized	Current mapped dimensions match the last stored synchronization baseline.
geometry newer	Spatial dimensions changed after the last known synchronization.
engineering newer	Engineering dimensions changed after the last known synchronization.
conflicting	Both sides changed, or available provenance is insufficient to establish authority.
unmapped	A valid explicit mapping is absent.

Conflict rule: do not use a conflicting or unmapped state as if it were automatically resolved. Determine the authoritative source, synchronize deliberately, then rerun the affected analysis.

CHAPTER 9**Standard operating procedure
SOP-04: validate, run and review an****9. Standard operating procedure SOP-04: validate, run and review an analysis****Procedure**

1. Select the analysis.
2. Review the input object and source references.
3. Click Validate. Validation should use the same parser/contract path as execution.
4. Correct all validation errors.
5. Click Run.
6. Keep the project/input unchanged while the run is active.
7. Review the normalized result, diagnostics/provenance, report and plot where available.
8. Check whether the result is fresh for the current input.
9. Compare outputs with expected physical behavior and independent hand checks for critical values.
10. Export evidence only after review.

9.1 Freshness protection

Completed results are bound to the canonical SHA-256 identity of the submitted analysis input. If the analysis kind or input changes, stale cached evidence is not intended to be presented as current. Rerun after meaningful input changes.

9.2 Abandon behavior

Abandoning an active run suppresses its pending result but does not necessarily force-terminate the underlying worker immediately. Conflicting mutation remains restricted until safe execution ownership is restored.

CHAPTER 10

Room and project verification

10. Room and project verification

10.1 Room volume

For rectangular room screening:

Volume = length x width x height

with dimensions in metres and volume in cubic metres.

10.2 Nominal supply ACH

ACH = supply airflow (m³/h) / room volume (m³)

This is a nominal bulk air-change calculation. It does not describe local velocity, mixing quality, short-circuiting, recovery distribution, or CFD flow fields.

10.3 Pressure-cascade checks

Project verification can evaluate explicit higher-pressure/lower-pressure room relationships using supplied room pressure evidence and a configured minimum differential.

Example input concept:

```
{
  "higher_pressure_room": "Process",
  "lower_pressure_room": "Preparation",
  "min_delta_pa": 10
}
```

CleanroomX does not invent a missing measured or calculated room pressure to force a pass.

10.4 Aggregate result states

State	Meaning
pass	Every included finding is evaluated and passing.
pass_with_unchecked	No evaluated finding fails, but one or more configured checks remain unevaluated.
fail	At least one evaluated finding fails.
not_checked	No applicable finding was evaluated.

Do not reduce these states to a simple green/red indicator when completeness matters.

CHAPTER 11

Particle decay and recovery screening

11. Particle decay and recovery screening

CleanroomX includes a first-order, well-mixed particle-decay model:

$$C(t) = C_0 \times \exp[-(ACH/60) \times \text{efficiency} \times t]$$

and the corresponding time-to-target relationship.

Typical CLI examples:

```
cleanroomx decay --initial 1000000 --ach 30 --minutes 10 --efficiency 1.0
cleanroomx recovery --initial 1000000 --target 100000 --ach 30 --efficiency 1.0
```

A dedicated recovery-test workflow can also evaluate measured time-series data against a target concentration and optional maximum recovery time.

Engineering limitations

The simple decay equation does not model local recirculation, active particle generation, surface deposition, leakage, imperfect mixing, thermal plumes, equipment wakes, door opening, operator movement, or spatial concentration gradients. Use measured testing and/or CFD when those effects matter.

CHAPTER 12

Psychrometrics, thermal and HVAC workflows

12. Psychrometrics, thermal and HVAC workflows

12.1 Steady airflow balance

CleanroomX uses the explicit balance:

$$\text{net surplus} = \text{supply} + \text{transfer_in} - \text{return} - \text{exhaust} - \text{transfer_out}$$

and:

$$\text{surplus margin} = \text{net surplus} - \text{required minimum surplus}$$

Positive surplus represents airflow available for exfiltration or another unmodeled outflow path; negative surplus implies an unmodeled inflow/infiltration path is required to close the balance.

12.2 Preliminary fan duty

The preliminary air-side pressure requirement is composed from explicit pressure components such as duct, coil, terminal/filter and other allowance inputs. Fluid power, shaft power and electrical input are calculated from airflow, pressure and configured efficiencies.

This calculation is a screening/design step. Final fan selection should use verified manufacturer performance data and project-specific allowances such as system effect, dirty-filter state, VFD/control losses, altitude and installation configuration.

12.3 Thermal uncertainty

The bounded thermal uncertainty workflow evaluates endpoint combinations of uncertain inputs and reports conservative intervals for derived psychrometric/load quantities. It is deterministic bounded screening, not a statistical Monte Carlo confidence model.

CHAPTER 13

Duct and airflow network analysis

13. Duct and airflow network analysis

CleanroomX includes workflows for duct pressure loss, branch networks, looped networks, variable-friction behavior and related provenance.

13.1 Engineering review points

For every network study, confirm:

- topology matches the actual design intent;
- units and section dimensions are correct;
- roughness/friction inputs are appropriate;
- fittings/minor-loss inputs are justified;
- boundary conditions are complete;
- supplied demands/flows are physically consistent;
- the solver converged or produced an explicitly valid solved state;
- the reported critical path/pressure loss is plausible;
- no presentation rounding is being used as the source of an engineering decision.

13.2 Numerical integrity

CleanroomX development places strong emphasis on deterministic solver state, full-precision engineering comparisons and provenance. Public report values may be rounded for readability; engineering decisions should remain tied to canonical numerical state wherever the workflow provides it.

CHAPTER 14**Fan/system operating-point studies****14. Fan/system operating-point studies**

CleanroomX can solve fan/system intersections from supplied fan-curve and system/network data, and supports speed/affinity studies, uncertainty studies, passive-network integration and loop-network integration.

14.1 Required engineering checks

- Confirm the fan curve belongs to the intended fan, speed and configuration.
- Confirm the operating region is within manufacturer-allowed limits.
- Confirm the system/network representation corresponds to the same air path.
- Review whether a unique intersection exists in the modeled range.
- Check that extrapolation or out-of-range behavior has not been silently assumed.
- Independently review the selected operating point for critical equipment selection.

14.2 Cross-study consistency

Where CleanroomX compares HVAC governing airflow with a fan/network operating airflow, the purpose is to detect contradictory studies, not to replace the underlying fan or HVAC calculation.

CHAPTER 15**Uncertainty and scenario studies****15. Uncertainty and scenario studies**

CleanroomX provides bounded uncertainty/scenario workflows for several model families. These results should be read as sensitivity envelopes under supplied bounds, not automatically as probabilistic confidence intervals.

For each uncertainty study, retain:

- base input;
- bound definition and rationale;
- source/provenance of uncertain values;
- worst/bounding result;
- decision threshold;
- whether the outcome is pass, fail, indeterminate, unknown or not checked where the workflow supports such states.

CHAPTER 16

Engineering consistency, dossier and assurance

16. Engineering consistency, dossier and assurance

16.1 Consistency checks

Consistency workflows compare related calculations or project requirements to detect internal contradictions. They do not create a new physical truth when the sources are wrong.

16.2 Engineering dossier

The dossier workflow composes source studies and their engineering evidence into a single traceable package. External file dependencies are fingerprinted so changed files are not silently mixed into an old run.

16.3 Design assurance

Design assurance can combine canonical design consistency, pressure-design consistency and versioned compliance evidence into a traceable result. Component and source digests support tamper detection and replay, but approval authority remains outside the software.

16.4 Assurance snapshots

Create a deterministic snapshot:

```
cleanroomx-assurance-snapshot create \  
  examples/design_assurance_demo.json \  
  design_assurance.snapshot.json
```

Verify it later:

```
cleanroomx-assurance-snapshot verify design_assurance.snapshot.json
```

A valid snapshot demonstrates internal content integrity and deterministic replay consistency under the verifier. It is not a digital signature and does not prove who authored or approved the source.

CHAPTER 17**ProofGraph - development preview****17. ProofGraph - development preview**

Status: DEVELOPMENT PREVIEW on current main; not part of the v0.102.1 stable baseline.

ProofGraph is the GUI-independent evidence model for requirement-to-evidence-to-finding-to-verdict traceability. It is not a second engineering solver.

Its documented schema is `cleanroomx.proofgraph`, version 1. The model includes requirement sets, evidence sources, design/calculation/simulation/commissioning/operational evidence types, provenance, confidence metadata, compliance checks/findings/verdicts, corrective actions and verification runs.

Key safety properties documented on current main include strict reference validation, acyclic provenance, explicit project identity, evidence-backed PASS findings, explicit non-pass states, and corrective actions that require approval rather than being applied automatically.

Where ProofGraph adapters exist, they are intended to reuse canonical CleanroomX analyses rather than duplicate equations.

CHAPTER 18**BIM / IFC interoperability****18. BIM / IFC interoperability**

CleanroomX supports BIM/IFC workflows with optional native IFC support through `ifcopenshell`.

18.1 BIM operating rules

- Preserve IFC GlobalId and source revision where available.
- Do not infer compliance from the presence of BIM metadata alone.
- Distinguish geometric dimensions from their extraction method/source.
- Review storey identity, placement, orientation, containment and room/device linkage.
- Treat imported design evidence as source evidence, not commissioning evidence.
- Revalidate mappings after replacing the IFC source.

18.2 Native IFC dependency

```
python -m pip install -e .[bim]
cleanroomx-ifc --help
```

CHAPTER 19

Persistence, autosave, recovery and revisions

19. Persistence, autosave, recovery and revisions

19.1 Atomic persistence

File-producing paths are designed to avoid partial in-place replacement where the shared persistence boundary applies. This is important for project files, reports, recovery artifacts and evidence packages.

19.2 Autosave and crash recovery

Recovery artifacts are separate from explicit project saves. On restart, inspect the recovery candidate and its source context before restoring it.

Recovery rule

Never assume the newest recovery artifact is automatically the authoritative project revision. Compare it with the last approved saved project and determine which work is valid.

19.3 Saved revisions

Use saved project revisions or your organization's version-control/document-control system before major edits, model migration, large geometry changes, rule-pack replacement or release upgrades.

CHAPTER 20

Diagnostics and health checks

20. Diagnostics and health checks

Run a project-wide health check:

```
cleanroomx-project-check project.cleanroomx.json
```

Project diagnostics can surface model, spatial, synchronization, input-validity, dependency, and stale-evidence problems without changing the design.

Recommended use:

- before issuing a report;
- after moving/renaming project folders;
- after replacing external source files;
- after large 2D/3D edits;
- before and after software-version upgrades;
- before creating a final bundle.

CHAPTER 21

Reporting and evidence handoff

21. Reporting and evidence handoff

21.1 Report types

Depending on workflow, CleanroomX can provide normalized result JSON, Markdown reports, portable/self-contained HTML engineering reports, run bundles, project bundles and assurance snapshots.

21.2 Report review checklist

Before release to a client/reviewer:

- [] Project and analysis names are correct.
- [] Software baseline is identifiable.
- [] Input/source revision is identifiable.
- [] Results are fresh for current inputs.
- [] Units are visible and correct.
- [] Acceptance criteria are explicit.
- [] not_checked, unknown or indeterminate states are not presented as PASS.
- [] Critical engineering values have an independent plausibility check.
- [] Assumptions and limitations are included.
- [] External dependencies have not changed since the run.
- [] The responsible engineer/approver is identified outside the software where required.

21.3 Portable project bundle

Use the project-bundle command for controlled handoff:

```
cleanroomx-project-bundle --help
```

Verify received bundles according to the command's verification workflow before treating them as a trusted copy of the sender's project package.

CHAPTER 22

Command-line reference

22. Command-line reference

Installed console commands in v0.102.1 include:

Command	Primary purpose
cleanroomx	Core CLI including basic room/decay/recovery functions.
cleanroomx-gui	Desktop application, demo and readiness check.
cleanroomx-project-run	Deterministic saved-project execution.
cleanroomx-project-bundle	Portable project evidence/handoff bundle.
cleanroomx-project-check	Project-wide diagnostics.
cleanroomx-ifc	BIM/IFC workflows.
cleanroomx-hvac	HVAC project analysis.
cleanroomx-recovery-test	Observed recovery qualification.
cleanroomx-qualification	Cleanroom qualification workflow.
cleanroomx-uncertainty	Verification uncertainty.
cleanroomx-thermal-uncertainty	Thermal uncertainty.
cleanroomx-psychrometric-uncertainty	Psychrometric uncertainty.
cleanroomx-duct-flow	Duct flow/path analysis.
cleanroomx-loop-flow	Looped network analysis.
cleanroomx-pressure-network	Room pressure/leakage network command present in current package entry points.
cleanroomx-loop-friction	Variable-friction loop study.
cleanroomx-fan-curve	Fan/system operating point.
cleanroomx-fan-uncertainty	Fan/system uncertainty.
cleanroomx-fan-speed	Fan speed/affinity study.
cleanroomx-fan-duct	Fan + duct network integration.
cleanroomx-fan-network	Fan + passive network integration.
cleanroomx-fan-loop	Fan + loop network integration.
cleanroomx-fan-loop-friction	Fan + variable-friction loop integration.
cleanroomx-fan-loop-friction-speed	Speed study for variable-friction fan/loop workflow.
cleanroomx-fan-loop-friction-uncertainty	Uncertainty for variable-friction fan/loop workflow.
cleanroomx-fan-loop-uncertainty	Fan/loop uncertainty.

Command	Primary purpose
cleanroomx-fan-loop-speed	Fan/loop speed study.
cleanroomx-damper-study	Damper/scenario study.
cleanroomx-dossier	Engineering dossier construction.
cleanroomx-consistency	Cross-module consistency.
cleanroomx-assurance-snapshot	Deterministic design-assurance snapshot create/verify.

Use --help on each command as the authoritative command-line syntax for the installed build.

CHAPTER 23**Industrial workflow examples****23. Industrial workflow examples****23.1 Example A - new cleanroom design package**

1. Establish project requirements and external acceptance criteria.
2. Create spatial layout and room identities.
3. Validate geometry.
4. Map engineering rooms.
5. Enter room airflows, ACH requirements and pressure targets.
6. Run room/project verification.
7. Build HVAC/thermal inputs.
8. Analyze duct/network pressure losses.
9. Evaluate fan/system operating point.
10. Run cross-module consistency.
11. Resolve contradictions.
12. Run diagnostics.
13. Create report/bundle/snapshot as applicable.
14. Perform independent engineering review.
15. Issue the controlled package through the organization's approval process.

23.2 Example B - design change after commissioning feedback

1. Duplicate/archive the approved project revision.
2. Record the change request and source measurement data.
3. Modify only the affected design inputs.
4. Review synchronization status.
5. Rerun affected analyses.
6. Check stale evidence warnings.
7. Compare before/after results.
8. Reissue only after engineering and QA approval.

23.3 Example C - external review handoff

1. Run `cleanroomx-project-check`.
2. Confirm all required runs are fresh.

3. Generate the engineering report.
4. Create the portable project bundle.
5. Provide the receiving party with the software baseline and verification instructions.
6. Receiver verifies the bundle/snapshot before review.

CHAPTER 24

Troubleshooting and failure handling

24. Troubleshooting and failure handling

Symptom	Required action
GUI does not start	Run <code>cleanroomx-gui --check</code> ; confirm the expected Python environment and Tk installation.
Input will not validate	Read the parser error; correct field names, types, units, required values and strict JSON. Do not bypass validation.
Result disappeared after editing	Expected stale-evidence protection. Rerun the analysis.
Save is refused because file changed externally	Do not overwrite blindly. Reopen/compare the external revision and reconcile changes.
Legacy file cannot overwrite itself after migration	Expected protection. Save the schema-v1 project to a new path first.
File-backed run reports changed dependency	Stop. Reconcile the source file revision and rerun.
Pressure/fan/network result is physically implausible	Review topology, boundary conditions, units, coefficients, curve data and solver status; independently calculate a sanity check.
Bundle/snapshot verification fails	Treat the artifact as unverified; obtain a fresh copy or identify the modification/version mismatch.
Recovered project differs from last approved save	Compare revisions; do not automatically promote recovery data to approved status.

24.1 Fail-safe rule

When the software reports invalid, stale, unknown, indeterminate, not checked, unresolved dependency, failed verification, or integrity mismatch, the operator must not convert that state into PASS by interpretation alone. Resolve the cause or document the exception through the project's formal engineering/quality process.

24.2 Deviation and exception control

If a project must proceed with an unresolved non-critical limitation, create a controlled deviation record before release. The record should identify:

- project and analysis identity;
- software/manual baseline;
- exact warning, limitation or unavailable check;
- engineering reason the condition does not invalidate the intended decision;
- compensating verification or independent calculation;
- affected deliverables;
- responsible reviewer and approver;

- required follow-up and closure condition.

A deviation record must not be used to relabel a failed calculation or violated acceptance criterion as PASS. It documents the disposition and approval of an exception; it does not alter the calculated evidence.

CHAPTER 25

Security and deployment

25. Security and deployment

25.1 Security principles

- Use trusted project and rule-pack sources.
- Restrict write access to controlled project directories.
- Do not treat SHA-256 as signer authentication.
- Keep the Python environment and dependencies controlled.
- Verify external file revisions before issuing evidence.
- Maintain backups independent of autosave/recovery.
- Review generated HTML/bundles according to organizational security policy before external distribution.

25.2 Deployment

For controlled use, prefer a reproducible installed package or pinned repository commit rather than an unpinned developer checkout. Validate the installed application with `cleanroomx-gui --check` and run the organization's acceptance/smoke test after deployment changes.

CHAPTER 26

Verification and validation strategy for organizations

26. Verification and validation strategy for organizations

CleanroomX includes substantial automated test evidence, but an organization using the tool in a controlled engineering process should define its own intended-use qualification proportional to risk.

A practical qualification package may include:

- installation qualification: version, environment, startup/readiness;
- operational qualification: representative room, HVAC, network, fan, report and recovery workflows;
- performance/fit-for-use checks: project-specific benchmark calculations and accepted reference cases;
- data-integrity checks: save/reopen, stale-result protection, external-write guard, bundle/snapshot verification;
- change-control regression: selected golden cases rerun after upgrades.

Change-impact / revalidation guide

Change	Minimum response before controlled reuse
Documentation-only change	Confirm no software behavior changed; retain approval record
Patch to report formatting only	Recheck affected report rendering and evidence identity
Solver/numerical implementation change	Rerun representative golden calculations and affected project analyses
Input schema or persistence change	Test open/save/migration, rollback path and historical project compatibility
GUI workflow change	Repeat readiness and representative operator workflow checks
Dependency/runtime upgrade	Repeat installation/readiness, CLI and GUI smoke checks; rerun project golden cases
Rule-pack/acceptance-criteria change	Reapprove criteria and rerun all affected verdicts
External source-file revision	Revalidate provenance and rerun dependent analyses

This is an organizational quality recommendation, not a claim that CleanroomX is already qualified for every regulated application.

CHAPTER 27

External standards and regulatory references

27. External standards and regulatory references

CleanroomX does not silently embed external acceptance criteria as universal truth. Where a project is governed by standards or regulations, the responsible engineer should obtain and control the applicable documents and encode only the approved project criteria.

Relevant official references include:

- ISO 14644-1:2015, *Cleanrooms and associated controlled environments - Part 1: Classification of air cleanliness by particle concentration*. ISO states this edition remains current following review/confirmation in 2021. Official information: <https://www.iso.org/standard/53394.html>
- ISO 14644-2:2015, *Cleanrooms and associated controlled environments - Part 2: Monitoring to provide evidence of cleanroom performance related to air cleanliness by particle concentration*. Official information: <https://www.iso.org/standard/53393.html>
- ISO 14644-3:2019, *Cleanrooms and associated controlled environments - Part 3: Test methods*. Official information: <https://committee.iso.org/standard/60598.html>
- EU GMP Annex 1 - Manufacture of Sterile Medicinal Products, revision published 25 August 2022; most provisions came into operation 25 August 2023, with point 8.123 deferred to 25 August 2024. Official European Commission information: https://health.ec.europa.eu/latest-updates/revision-manufacture-sterile-medicinal-products-2022-08-25_en

Do not copy a limit from a secondary source into a CleanroomX rule pack without confirming applicability, edition, facility type, operating state, measurement method and contractual/regulatory context.

CHAPTER 28

Controlled release checklist

28. Controlled release checklist

Use this checklist before issuing a CleanroomX engineering package.

28.1 Engineering traceability matrix

The released package should allow a reviewer to move from requirement to source input, calculation, result and approval without relying on memory.

Traceability element	Minimum evidence
Requirement	Controlled requirement identifier, source document/rule pack and revision
Source input	Project field or external file reference, unit, revision/hash where applicable
Calculation	Analysis kind, software version/commit and execution evidence
Result	Normalized result plus status/completeness and critical numerical values
Provenance	Input digest, dependency identity and freshness/diagnostic evidence
Review	Independent plausibility/consistency check and disposition of exceptions
Issued artifact	Report/bundle/snapshot identity tied to the approved project revision
Approval	Authorized human approval in the organization's controlled process

If any row cannot be reconstructed for a critical requirement, the package is not yet ready for controlled issue.

Baseline

- [] Approved CleanroomX version/commit recorded.
- [] Supported Python/runtime confirmed.
- [] Project revision is saved and protected from accidental overwrite.

Inputs

- [] Room identity and geometry reviewed.
- [] Units checked.
- [] External source files and manufacturer data revisions recorded.
- [] Acceptance criteria/rule packs reviewed by the responsible engineer.
- [] Required mappings are complete.

Execution

- [] Inputs validated.
- [] Required analyses rerun after the last relevant change.

- ☐ Project diagnostics reviewed.
- ☐ No stale result is being issued.
- ☐ Solver/result status is acceptable and understood.

Engineering review

- ☐ Critical outputs independently sanity-checked.
- ☐ Cross-module contradictions resolved.
- ☐ Unchecked/unknown/indeterminate outcomes dispositioned.
- ☐ Assumptions and limitations documented.

Evidence

- ☐ Report/bundle/snapshot generated from the approved revision.
- ☐ Integrity verification completed where applicable.
- ☐ Final approval/signature performed through the organization's authorized process.

CHAPTER —

Appendix A - recommended project folder structure

Appendix A - recommended project folder structure

```
Project_Name/  
  00_Admin/  
    project_basis.pdf  
    change_log.md  
  01_Inputs/  
    requirements/  
    manufacturer_data/  
    measurements/  
    bim/  
  02_CleanroomX/  
    project.cleanroomx.json  
    revisions/  
  03_Reports/  
    engineering_report.html  
    dossier.md  
  04_Evidence/  
    bundles/  
    assurance_snapshots/  
    hashes/  
  05_Review/  
    comments/  
    approvals/
```

The exact structure may be changed to match the organization's document-control system. The principle is to separate controlled source inputs, working project files, issued reports and review/approval records.

CHAPTER —

Appendix B - operator quick reference

Appendix B - operator quick reference

Start and check

```
cleanroomx-gui --check  
cleanroomx-gui --demo
```

Run project diagnostics

```
cleanroomx-project-check project.cleanroomx.json
```

Execute a saved project

```
cleanroomx-project-run project.cleanroomx.json
```

Project bundle

```
cleanroomx-project-bundle --help
```

Assurance snapshot

```
cleanroomx-assurance-snapshot create INPUT.json SNAPSHOT.json  
cleanroomx-assurance-snapshot verify SNAPSHOT.json
```

IFC

```
cleanroomx-ifc --help
```

HVAC / network / fan families

```
cleanroomx-hvac --help  
cleanroomx-duct-flow --help  
cleanroomx-loop-flow --help  
cleanroomx-fan-curve --help  
cleanroomx-fan-network --help  
cleanroomx-fan-loop --help
```

CHAPTER —

Appendix C - glossary

Appendix C - glossary

ACH - air changes per hour; in CleanroomX room screening, supply airflow divided by room volume. Atomic save - persistence pattern that writes a validated replacement and commits it as a unit rather than truncating the live file in place. Canonical state - the internal engineering/numerical state used as the authoritative calculation basis before presentation rounding. Development preview - capability present on current main but not part of the controlled stable release baseline. Evidence freshness - confirmation that a result still corresponds to the current input/dependency revision. IFC GlobalId - stable IFC entity identity used for BIM traceability when available. Indeterminate - evaluated state where available bounded evidence does not support a single pass/fail conclusion. Not checked - a known criterion was not evaluated. ProofGraph - development evidence graph linking requirements, evidence, checks, findings and verdicts. Provenance - information identifying where an input/result came from and how it was derived. Rule pack - explicit versioned set of criteria supplied to a compliance-evidence workflow. Stale evidence - a result that no longer corresponds to the current analysis input/dependency state. TAB - testing, adjusting and balancing.

CHAPTER —

Appendix D - controlled source map

Appendix D - controlled source map

Repository documents that should be consulted for implementation-level detail include:

- README.md
- ARCHITECTURE.md
- TEST_EVIDENCE.md
- VALIDATION.txt
- CHANGELOG.md
- SECURITY.md
- DEPLOYMENT.md
- MIGRATIONS.md
- ROLLBACK.md
- docs/APPLICATION_GUI.md
- docs/LAYOUT_2D_3D.md
- docs/PROJECT_DIAGNOSTICS.md
- docs/ENGINEERING_DOSSIER.md
- docs/DESIGN_ASSURANCE.md
- docs/ASSURANCE_SNAPSHOTS.md
- docs/PROOFGRAPH.md (development preview)
- workflow-specific engineering documents under docs/

When this manual conflicts with the installed software behavior, stop and resolve the discrepancy against the controlled software baseline before issuing engineering evidence.